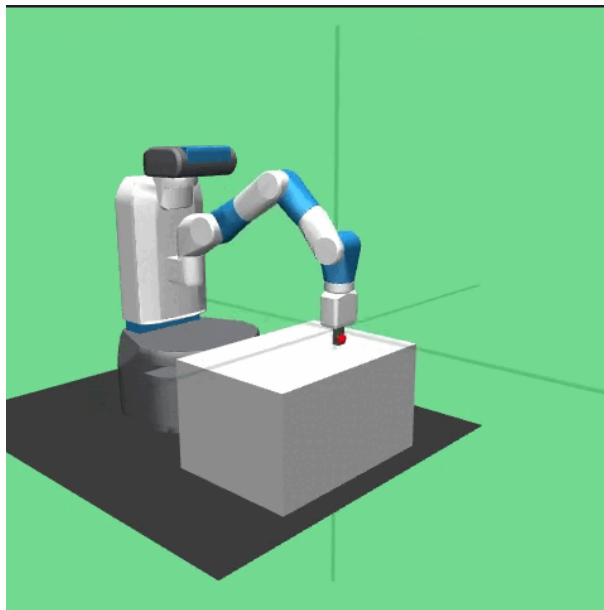


Imitation Learning on the FetchReach Environment



Comparison of the DAgger Algorithm with Three Types of Experts

Talaoubrid Nacim
Couedel Romane
Mohamed El Oualid Boudemagh

Sorbonne Université
November 28, 2025

github : <https://github.com/Ouabm/Imitation-learning-project-.git>

Abstract

This report presents the implementation of the DAgger algorithm on the **FetchReach-v4** robotic environment. Three types of experts are evaluated: a human operator, a PID-based analytic controller, and a reinforcement learning (RL) policy. We describe the experimental setup, the metrics used to evaluate the quality of demonstrations, and the performance obtained through behavioral cloning and DAgger. Special attention is given to the effect of expert quality on the learned policy and the importance of dataset aggregation in mitigating distribution shift.

Contents

1	Introduction	2
2	FetchReach Environment	2
3	Dagger Algorithm	3
4	Evaluation Metrics	4
4.1	Final Distance to the Target	4
4.2	Episode Reward	5
5	Global Results	5
6	Alternative Experts	7
6.1	PID Expert	7
6.2	Reinforcement Learning Expert	7
7	Final table	7
8	Conclusion	8

1 Introduction

Imitation Learning aims to learn a control policy directly from expert demonstrations. A straightforward approach is Behavioral Cloning (BC), where a supervised model is trained to imitate expert actions. However, BC typically suffers from *distribution shift*: the agent encounters states during execution that were absent in the expert dataset, leading to cascading errors.

To address this limitation, we implement DAgger (Dataset Aggregation), introduced by Ross et al. (2011). DAgger progressively enriches the dataset by querying the expert on states visited by the learned policy, increasing robustness and stability.

We test the algorithm on the **FetchReach-v4** environment, a 3D reaching task involving a robotic arm. Three expert types are compared to analyze the influence of expert quality on imitation performance.

2 FetchReach Environment

The **FetchReach** environment is part of the **Gymnasium Robotics** suite and simulates the motion of a 7-DoF Fetch robotic manipulator in a MuJoCo-based physics engine. The objective is to control the end-effector so that it reaches a randomly sampled 3D target position. Although the task is mechanically simple, it is a standard benchmark for continuous-control algorithms because it combines high-dimensional states with sparse rewards.

Action Space. At each timestep, the agent outputs a 4-dimensional continuous action:

$$a = [\Delta x, \Delta y, \Delta z, \Delta \text{gripper}]$$

where the first three components correspond to Cartesian displacements of the end-effector, and the fourth controls the gripper opening. In **FetchReach**, the gripper action has no effect on the task but is maintained for consistency with other Fetch tasks.

Observation Space. The observation returned by the environment is a dictionary containing:

$$s = \{\text{robot observation}, \text{achieved_goal}, \text{desired_goal}\}$$

The **observation** field includes the robot’s joint positions, joint velocities, and the spatial position of the end-effector. The *achieved_goal* is the current end-effector position, while the *desired_goal* is the randomly sampled target.

Reward Structure. **FetchReach-v4** uses a binary sparse reward:

$$r_t = \begin{cases} 0 & \text{if } d_f = \|p_{ee} - p_{\text{target}}\|_2 < 0.05, \\ -1 & \text{otherwise.} \end{cases}$$

The agent receives a penalty of -1 at each timestep until the target is reached, leading to episodes where performance is measured by the time needed to achieve a successful reach. A success event is registered when the final distance d_f is below the 5 cm threshold.

Motivation. The `FetchReach` environment provides a high-dimensional and continuous control setting, with a rich observation vector combining joint states, velocities, and Cartesian end-effector information. Its continuous action space further increases the complexity of the control problem. For these reasons, we selected this environment as a representative benchmark to evaluate our learning algorithms.

As a first baseline, we trained an off-policy reinforcement learning model (Soft Actor-Critic, SAC) on `FetchReach`. The learning curve reporting the episode return as a function of training iterations is shown in Figure 1.

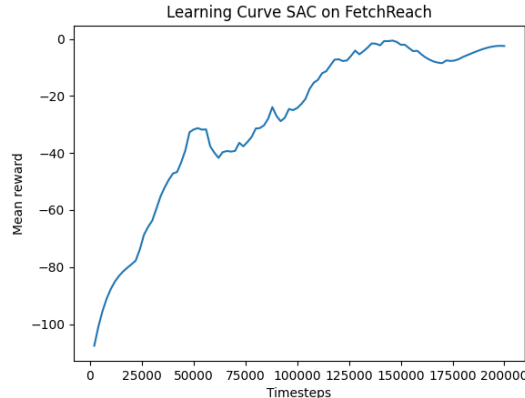


Figure 1: SAC learning curve on `FetchReach` showing the evolution of the average reward over training steps.

The curve shows that, despite the simplicity of the reaching task, the RL agent requires a substantial number of interaction steps before converging. This behavior aligns with expectations: the sparse reward structure makes exploration difficult, and the agent receives only limited feedback until it becomes sufficiently skilled to consistently reach the target.

Because of this slow convergence, we turn to imitation learning techniques such as Behavioral Cloning (BC) and DAgger. These methods leverage expert demonstrations to reduce the exploration burden and significantly accelerate training by replacing the sparse reward signal with direct supervision on the desired actions.

3 DAgger Algorithm

The Dataset Aggregation (DAgger) algorithm addresses a key limitation of Behavioral Cloning (BC): the distribution shift between expert demonstrations and the states visited by the learned policy. While BC trains a policy solely on the expert’s trajectories, it fails when the policy encounters states not present in the initial dataset. DAgger explicitly mitigates this mismatch by iteratively collecting expert labels on the states induced by the current policy.

The procedure can be described as follows:

1. Initialize an empty dataset and a starting policy, which can be any policy from the chosen policy class.
2. For each iteration of training:

- (a) Construct a mixed policy by combining the current learned policy with the expert policy. Initially, the expert dominates, but over iterations the learned policy gradually takes control.
 - (b) Execute the mixed policy in the environment to collect trajectories of states visited by the policy.
 - (c) For every state encountered during these trajectories, query the expert to obtain the correct action for that state.
 - (d) Retrain or fine-tune the policy on the expanded dataset to improve its performance.
3. After completing all iterations, select the best-performing policy based on validation performance.

To gradually shift control from the expert to the learned policy, DAgger uses a mixing parameter β_i that determines the probability of applying the expert action at iteration i . We use a linear decay schedule to ensure a smooth transition:

$$\beta_i = \max\left(0, 1 - \frac{i}{N}\right) \quad , \quad \pi_i = \beta_i \pi^* + (1 - \beta_i) \hat{\pi}_i$$

where N is the total number of DAgger iterations, π^* is the expert policy and $\hat{\pi}_i$ is the fine-tuned policy. Early iterations rely more heavily on the expert, while later iterations encourage the policy to act autonomously, all while benefiting from expert corrections on the states it actually encounters. This iterative feedback loop enables DAgger to achieve significantly better robustness and generalization than standard Behavioral Cloning.

Human Expert In our implementation, DAgger relies on a human expert to provide corrective actions during the aggregation phase. The expert controls the end-effector of the Fetch robot manually using the keyboard:

- $\leftarrow, \rightarrow$: motion along the X-axis
- $\uparrow, \downarrow$: motion along the Y-axis
- W, A : motion along the Z-axis

4 Evaluation Metrics

4.1 Final Distance to the Target

$$d_f = \|p_{ee} - p_{\text{target}}\|_2$$

This metric quantifies reaching precision.

4.2 Episode Reward

Since the environment provides a sparse reward at every timestep, the episode return is defined as:

$$R = \sum_{t=1}^T r_t.$$

In **FetchReach**, an episode contains $T = 100$ timesteps by default. Because the step reward is -1 until the goal is reached and 0 once the target is achieved, the episode return equals the negative number of timesteps spent away from the goal (with a minimum of $-T$). Consequently the return lies in the interval $[-100, 0]$. Examples and interpretation:

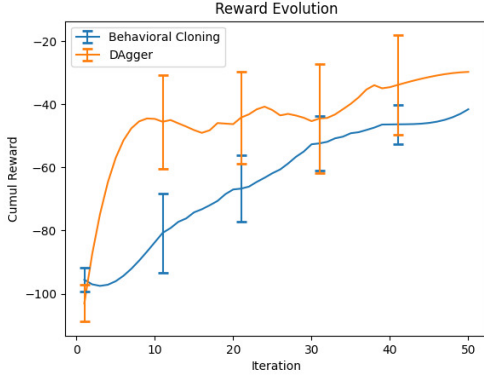
- $R = -100$ indicates a very poor outcome (the agent never reached the goal during the episode).
- $R = -5$ means the agent reached the goal after 5 timesteps (good performance — the agent is fast).
- $R = 0$ indicates immediate success or that the agent was already at the goal at the first evaluated timestep.

This sparse, per-step penalty strongly encourages speed: policies that reach the target in fewer steps obtain better (less negative) returns. Therefore, a model producing returns close to 0 (or in the range $[-5, 0]$) can be considered performant and fast, whereas returns near $-T$ (e.g. -100) reflect poor behavior.

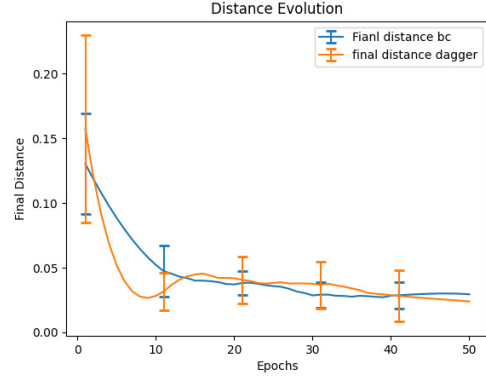
So the final distance metric indicates whether the robot successfully reaches the goal, while the reward metric provides insight into how quickly the robot reaches it.

5 Global Results

We compare the performance of two approaches on the **FetchReach** task: Behavioral Cloning (BC), DAgger. Our evaluation focuses on the final distance to the target and the reward, over multiple evaluation episodes. We first trained a Behavioral Cloning model using a large expert dataset of approximately 10^4 state-action pairs. As a result, performance plateaus quickly, and adding more data does not improve generalization. We also trained DAgger starting with *no* initial dataset. At the first iteration, the policy is purely controlled by the human expert, and at each subsequent iteration, the mixing coefficient decreases so that the learned policy gradually replaces the expert. We report 95% confidence intervals on the average final distance and reward every 10 epochs.



(a) reward comparaison



(b) Distance comparaison

Reward Analysis. In Figure 2a, we observe that after only ten iterations, the DAgger policy already converges toward a significantly improved reward of approximately -50 . In contrast, the Behavioral Cloning baseline stabilizes around -90 , confirming our hypothesis that interactive expert feedback enables the learner to rapidly correct its own errors and recover from suboptimal trajectories. The progressive aggregation of expert-labeled states allows DAgger to adapt to the distribution of states it effectively encounters, leading to a much faster and more stable convergence.

However, Figure 1 shows that the SAC agent, after full training, reaches a reward near -5 , substantially better than both imitation-learning approaches. This performance gap can be explained by the nature of the expert: since the expert is a human operator, it is inherently difficult to provide consistently optimal demonstrations in a continuous, high-precision control environment. SAC, through extensive exploration and optimization, eventually discovers trajectories that are smoother and more efficient than those demonstrated by the human expert.

Final Distance Analysis. The evolution of the final distance to the target shows a different trend from the reward curves. As illustrated in Figure 2b, DAgger converges slightly faster than Behavioral Cloning during the first iterations, thanks to the corrective feedback provided by the expert on states that the learned policy would not encounter in the supervised dataset alone. However, after a small number of epochs, both methods reach a similar final distance, indicating that once the basic reaching behaviour is learned, additional expert corrections bring limited improvements for this relatively simple task.

When comparing these results with the SAC agent, we observe that the final distance achieved after full reinforcement-learning training is approximately the same as that obtained by the imitation-learning methods. This suggests that all approaches ultimately learn to reach the target reliably, even though they differ substantially in sample efficiency and reward optimization.

Overall, our experiments confirm that DAgger converges significantly faster than SAC in terms of learning a policy that consistently reaches the goal, and it performs marginally better than Behavioral Cloning despite the absence of an initial dataset. However, when considering reward optimization, DAgger does not reach the target as rapidly as SAC, which benefits from extensive exploration and reward shaping. Nevertheless, all methods eventually succeed in achieving the task, demonstrating that imitation learning can be a highly efficient alternative to reinforcement learning for reach-based control problems.

6 Alternative Experts

Training a policy using a human expert is costly and time-consuming. Human demonstrations require constant attention and physical interaction, making large-scale data collection impractical. Moreover, human performance is inherently non-deterministic: trajectories can vary over time, and different humans may produce slightly different behaviors. This variability can introduce noise into the dataset and affect the learning process.

To address these limitations, we implement two alternative types of experts that provide more consistent and scalable supervision: a PID controller that mimics basic human behavior, and a pre-trained RL model.

6.1 PID Expert

The analytical expert relies on a PID controller:

$$u(t) = K_p e(t) + K_i \int e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

where $e(t)$ is the error between the current end-effector position and the target. The PID controller is deterministic and stable when properly tuned, providing precise corrective actions without human intervention.

6.2 Reinforcement Learning Expert

For the RL expert, we use the SAC model that we previously trained on the FetchReach-v4 environment. As we observed during its initial training, this model is very precise and consistently reaches the target very quickly. It therefore provides a reliable and deterministic source of supervision for the DAgger algorithm, without requiring further human intervention.

7 Final table

		SAC	Behavioral Cloning	DAGGER		
				PID	Human	Model
10 Epochs	Reward	-99.02	-90.47	-29.38	-32.74	-85.43
	Final Distance	0.302	0.07	0.02	0.04	0.13
Max training	Reward	-2.59	-41.91	-15.51	-27.78	-3.59
	Final Distance	0.01	0.03	0.008	0.03	0.02
	Human Cost	∅	High	Low	Medium	Low
	Robot Learning Cost	High	Medium	Low	Low	Low

Table 1: Comparison of SAC, BC, and DAgger with different experts including human and robot learning costs.

Analysis. The final table provides a comparative overview of the performance of SAC, Behavioral Cloning (BC), and DAgger using different experts, evaluated both after 10 epochs and at the end of full training. SAC achieves the highest cumulative rewards and the smallest final distances once fully trained, demonstrating its ability to reach the target efficiently and consistently. DAgger shows clear advantages over BC during the early stages of training: with the PID and human experts, it converges faster and accumulates higher rewards, highlighting the benefit of iterative expert corrections. Interestingly, after sufficient training, the final distances of all methods become comparable, suggesting that all approaches can ultimately guide the robot to the goal. The main difference lies in early learning dynamics, where the choice of expert strongly influences how quickly the policy improves and how efficiently the robot reaches the target during initial training.

8 Conclusion

In this project, we evaluated several imitation learning strategies on the FetchReach-v4 environment, with a particular focus on the DAgger algorithm and its dependence on expert quality. Our experiments confirm that DAgger effectively mitigates distribution shift and significantly outperforms standard Behavioral Cloning during early training stages. By continuously querying the expert on states induced by the learned policy, DAgger benefits from richer and more relevant data, enabling faster convergence and more stable behaviors.

The choice of expert plays a central role in the final performance. While human demonstrations provide intuitive guidance, they introduce inconsistencies and require substantial time investment. In contrast, analytical (PID) and reinforcement learning (SAC) experts offer deterministic, repeatable demonstrations that lead to more reliable policies. SAC, in particular, produces expert trajectories that are smoother and more efficient than those provided by humans, and serves as a strong upper bound for imitation learning performance.

Despite these differences in early convergence and reward optimization, all methods ultimately learn to reach the target reliably, indicating that FetchReach remains a manageable control task once the essential dynamics are understood. Reinforcement learning still achieves the best performance when training time is unlimited, but imitation learning with the right expert achieves competitive results in a fraction of the samples.

Future work could investigate more challenging manipulation tasks, evaluate the robustness of imitation learning approaches under noisy or imperfect experts, and explore transferring the learned policies to real robotic arms. These directions would help better understand the scalability and real world applicability of DAgger based approaches in robotics.